Lakshyam: a story-driven life simulation for financial literacy

Lakshyam ("goal" in Telugu and Sanskrit). A browser game where a student lives one whole life and sees what money choices do over time.

1. Problem statement and our understanding

The brief asks for a game that teaches financial literacy to Class 11–12 students from underserved communities. It should be simple, relatable and led by a persona that adapts to the player's age and context. It should be ready for Indian languages, test learning through play, have an intuitive interface and be usable by NGOs. It has to cover four areas: planning and budgeting, saving and managing money, understanding investing, and staying financially safe. Two guardrails: *educate, do not recommend investments*, and *simulations are assumptions, not promises*.

Our reading of the problem is that these students do not mainly lack information. They lack a safe place to make money mistakes before the mistakes are real. A 17-year-old in a Telangana village will soon be offered a phone at "only ₹699 a month", a betting app, a loan app, or ₹5,000 to "rent" a bank account. Most existing material is a lecture or a quiz, in English, with urban examples and no consequences. Real consequences arrive years later, so the link between a choice and its cost is never felt. We designed around four points that follow from this:

- **Their world, not ours.** The first persona is a tenant-farming family with no bank account and an open credit page at the kirana shop. The money comes from pocket money, cotton picking and a first factory job, not salaries and stock portfolios.
- **Delay made visible.** A choice made as a teenager must be able to come back as an adult, and the game must say so.
- **Fair to the player.** Poverty is never the player's fault. Shocks happen to everyone; preparation only makes them smaller, and every bad path has a way back.
- **Measurable.** A teacher or NGO should get a per-lesson report, not just a score.

2. Idea

The player picks a storyline and a dream: a pucca house, a small business, a sister's debt-free nursing degree, a vehicle that earns, or a secure old age for their parents. Then they live a whole life, one year per turn, in about 20–25 minutes. Each year has the same rhythm: income arrives, the player splits it into jars (needs, wants, emergency, goal, and later invest), may spend the wants jar in a small shop, then answers one to three story cards. A fixed curriculum of 24 lessons (L01–L24) runs from "needs vs wants" in childhood to "pension and nominees" later in life, across the four areas. The engine guarantees that every life covers every lesson inside its age window. Randomness picks which version of a lesson appears, never whether it appears.

Every lesson card follows the same loop: a situation, a choice, an outcome with a real number, and a two-line "why". Later, a teach-back card has a friend or sibling ask the player's advice on the same idea. The story keeps it from feeling like homework. The grandmother, Ammamma, wears thick glasses and calls the player *kanna*. The best friend Chintu always takes the shortcut, and his life runs alongside as a rival. A smooth talker returns every few years with a bigger scheme. A family secret about land lost to compound interest is revealed just as the player is about to buy land as an adult.

Mechanics that carry the learning

Mechanic	What the player does	Teaches
Echoes	A short-term choice leaves a faint hint. Years later the consequence card says where it came from: "an echo from age 15".	Delayed cost of EMIs, skipped insurance, no emergency fund
Ammamma's glasses	A limited number of charges reveal what phrases like "zero down payment" or "guaranteed double" really mean. Lessons already learnt are revealed for free.	Reading offers and fine print
Documents	Read a sale deed, loan agreement, policy or chit form and tap the suspicious line, e.g. a missing encumbrance certificate. A missed problem returns later.	Verifying before signing
Sahukar's chair	Sit at the moneylender's desk for a day and choose loan offers to maximise his profit, then see what each loan did to the borrower.	Monthly payment vs total cost vs yearly rate
Parallel Life	At the end, change one past decision. The same seed replays the life, and both lives are shown side by side.	Cause and effect, separated from luck

3. High-level architecture

The game is a static React + Vite web app for a laptop browser, built in three parts with a fixed written contract between them, so content, rules and screens can be built in parallel and swapped independently.

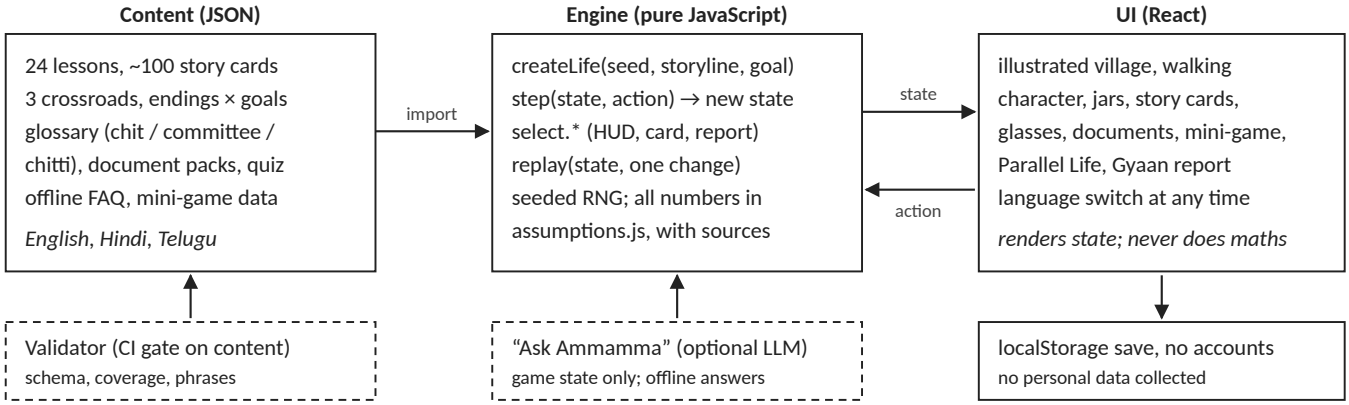


Figure 1. Three parts with one contract. The UI sends actions; the engine returns a new state; content is data only.

One turn (one year of life)

```
new year → budget → shop → card ⇄ outcome →
[mini-game] → year end → … → ending → Gyaan
report → Parallel Life
```

At the start of each year the engine pays income and automatic EMIs and premiums, grows holdings with seeded returns, applies inflation, and queues any consequence cards due this year. Shocks such as a flood or hospital bill are paid by insurance first, then the emergency jar, then other savings, and only then by forced borrowing from the costliest lender the player can reach. The outcome screen shows that breakdown, so the player sees exactly why preparation mattered.

Design decisions

- **One source of truth for money.** Every rate, premium, wage and goal cost lives in one assumptions file, each with a comment on what it is based on. The same file feeds an in-game "How this world works" panel. Values are illustrative and returns are shown as ranges.
- **Seeded and reproducible.** The same seed gives the same life. That is what makes Parallel Life fair, and lets a teacher replay a class demo.
- **Works offline.** The AI mentor is optional. If it fails or takes longer than 8 s, the closest offline answer is shown. It never changes numbers or events.
- **Safe by design.** No real brands or products. Illegal paths (mule, betting, Ponzi recruiting) show consequences, never methods, and always offer a way back, such as the 1930 cyber-crime helpline.

4. Planned testing methodology

Layer	How we test it	Pass condition
Engine rules	Unit tests with Node's built-in runner: loan EMIs against closed-form formulas, fund conservation on every transaction, shock payment order, seed reproducibility, Parallel Life replay.	All tests green on every push to main
Content	A validator script: schema, ids, lesson age windows, every scheduled card exists, glasses phrases appear word for word in each language, glossary and cast tokens resolve.	0 errors before content is merged
Whole-life simulation	Play thousands of seeded lives automatically with three policies: always-good, random and always-bad choices, across all five goals.	24/24 lessons in every life; no stuck states; good play never gets the legal-trouble ending
Balance	From the same runs, look at ending distributions, final debt and the emergency fund at the end of life, and tune the assumptions. An early run showed good play still ending with no savings cushion, which we are fixing.	Endings match intent: preparation visibly pays off, but nothing is guaranteed
Interface	Playwright browser tests at 1024×768, 1280×720 and 1366×768: a full life, keyboard use, Escape and focus, no console errors, no horizontal scroll.	Full life playable at all three sizes
Language	Hindi and Telugu checked by native speakers for everyday words; sentences kept to 20 words or fewer.	No English fallback on any screen
Learning	A 5-question check before and after each life, plus the end-of-life Gyaan report (applied / seen / missed per lesson). If time allows, short think-aloud play sessions with a few students.	Post-quiz score above pre-quiz; time per life under 25 min

Status at M1: the engine and its tests (47 passing), the content validator and the complete English village storyline are on main. A first sweep of 60 simulated lives ran with no errors and full lesson coverage. The full-game interface, Hindi and Telugu, and the mentor are the next steps.